

Christian_Herrera_HW9

April 6, 2026

1 Homework 9: Pandas (Questions)

2 Assignment Submission Guidelines

Please follow the guidelines below for submitting your assignment:

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM next Tuesday (03/31)**.
- Late submissions will not be accepted unless prior arrangements have been made by the TAs.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Please name your files as follows: Lastname_Firstname_AssignmentName
- Example: Alex_John_HW9.ipynb, and Alex_John_HW9.pdf.

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, please contact the TAs immediately **before the deadline** to avoid penalties.

3 Questions

```
[61]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data = {
    'employee_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Evan', 'Fiona', 'George', 'Hannah', 'Ian', 'Jenna'],
    'department': ['HR', 'Engineering', 'Engineering', 'Marketing', 'HR', 'Finance', 'IT', 'Marketing', 'Finance', 'IT'],
    'join_date': ['2020-01-10', '2019-04-23', '2021-07-04', '2018-02-11', '2022-05-19', '2019-11-09', '2020-06-01', '2021-03-15', '2019-08-22', '2020-11-03'],
```

```

        'salary': [70000, 85000, 80000, 75000, 68000, 72000, 78000, 74000, 77000, 79000],
        'projects_completed': [5, 9, 7, 4, 2, 6, 8, 3, 10, 5]
    }

employees = pd.DataFrame(data)

```

4 Question 1 (10 Points)

How many employees are there in the Engineering department?

Filter the DataFrame to only include employees from the Engineering department and count them.

```

[62]: # Keep only engineers.
engineering_employees = employees[employees["department"] == "Engineering"]

# How many?
engineering_count = engineering_employees.shape[0]

print(engineering_count)

```

2

5 Question 2 (10 Points)

What is the average salary of employees in the Marketing department?

Filter the DataFrame for employees in the Marketing department and calculate the average salary.

```

[63]: # Just marketing.
marketing_employees = employees[employees["department"] == "Marketing"]

# Average pay?
marketing_average_salary = marketing_employees["salary"].mean()

print(marketing_average_salary)

```

74500.0

6 Question 3 (10 Points)

Who is the employee with the highest number of projects completed? What department do they work in?

Identify the employee with the maximum projects completed and retrieve their name and department.

```
[64]: # The superstar employee.
superstar = employees["projects_completed"].max()

top_employee = employees[employees["projects_completed"] == superstar]

# Name and department only.
name_department = top_employee[["name", "department"]]

print(name_department)
```

```
name department
8 Ian      Finance
```

7 Question 4 (10 Points)

Create a new column in the DataFrame named year_joined that contains only the year from join_date.

Extract the year from join_date and create a new column.

```
[65]: # Turn join_date into dates. Could have been done with a string slice.
employees["join_date"] = pd.to_datetime(employees["join_date"])

# Year?
employees["year_joined"] = employees["join_date"].dt.year

# Showing the relevant data.
print(employees[["name", "join_date", "year_joined"]])
```

	name	join_date	year_joined
0	Alice	2020-01-10	2020
1	Bob	2019-04-23	2019
2	Charlie	2021-07-04	2021
3	Diana	2018-02-11	2018
4	Evan	2022-05-19	2022
5	Fiona	2019-11-09	2019
6	George	2020-06-01	2020

7	Hannah	2021-03-15	2021
8	Ian	2019-08-22	2019
9	Jenna	2020-11-03	2020

8 Question 5 (10 Points)

Find the total number of projects completed by each department.

Group the DataFrame by department and sum up the projects_completed

```
[66]: # Add up projects by department.
projects_by_department = employees.groupby("department")["projects_completed"].
    ↪sum()

print(projects_by_department)
```

```
department
Engineering    16
Finance         16
HR              7
IT             13
Marketing        7
Name: projects_completed, dtype: int64
```

9 Question 6 (10 Points)

Check for Duplicate Rows in the DataFrame

Determine if there are any duplicate rows in the employees DataFrame. Consider all columns for identifying duplicates.

```
[67]: # Check for repeated rows.
duplicate_rows = employees.duplicated()

print(duplicate_rows)

# Count them too.
print(duplicate_rows.sum())
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
dtype: bool
0
```

10 Question 7 (10 Points)

Find Missing Values

Check for missing values in the DataFrame. Report which columns have missing values and the count of missing values in each of those columns.

```
[68]: # Count missing stuff by column.
missing_values = employees.isnull().sum()

# Only show columns that actually have missing values.
missing_values = missing_values[missing_values > 0]

print(missing_values)
```

```
Series([], dtype: int64)
```

11 Question 8 (10 Points)

Replace Missing Values with Zero

Replace any missing values in the DataFrame with zero. Note: If the original DataFrame doesn't have missing values, introduce a few for demonstration.

```
[69]: # Make a separate demo dataframe.
employees_demo_missing = employees.copy()
```

```
# Add a couple missing values on purpose.
employees_demo_missing.loc[2, "salary"] = None
employees_demo_missing.loc[7, "projects_completed"] = None

# Fill the missing spots.
employees_filled = employees_demo_missing.fillna(0)

print(employees_filled)
```

	employee_id	name	department	join_date	salary	projects_completed	\
0	101	Alice	HR	2020-01-10	70000.0	5.0	
1	102	Bob	Engineering	2019-04-23	85000.0	9.0	
2	103	Charlie	Engineering	2021-07-04	0.0	7.0	
3	104	Diana	Marketing	2018-02-11	75000.0	4.0	
4	105	Evan	HR	2022-05-19	68000.0	2.0	
5	106	Fiona	Finance	2019-11-09	72000.0	6.0	
6	107	George	IT	2020-06-01	78000.0	8.0	
7	108	Hannah	Marketing	2021-03-15	74000.0	0.0	
8	109	Ian	Finance	2019-08-22	77000.0	10.0	
9	110	Jenna	IT	2020-11-03	79000.0	5.0	

	year_joined
0	2020
1	2019
2	2021
3	2018
4	2022
5	2019
6	2020
7	2021
8	2019
9	2020

12 Question 9 (10 Points)

Group By Department and Calculate Average Salary

Use the groupby method to group employees by their department and calculate the average salary in each department.

```
[70]: # Average salary for each department.
department_salary_average = employees.groupby("department")["salary"].mean()
```

```
print(department_salary_average)
```

```
department
Engineering    82500.0
Finance         74500.0
HR              69000.0
IT              78500.0
Marketing       74500.0
Name: salary, dtype: float64
```

13 Question 10 (10 Points)

Assign a Performance Category to Each Employee

Write a function that takes an employee's number of projects completed as input and returns 'High', 'Medium', or 'Low' based on the following criteria: 'High' if the number of projects completed is greater than 7. 'Medium' if the number of projects completed is between 4 and 7 (inclusive). 'Low' if the number of projects completed is less than 4. Apply this function to the `projects_completed` column to create a new column named `performance`.

```
[71]: # Sort people by project count.
def get_performance_level(project_count):
    if project_count > 7:
        return "High"
    elif project_count >= 4:
        return "Medium"
    else:
        return "Low"

# Add the new column.
employees["performance_level"] = employees["projects_completed"].
    ↪ apply(get_performance_level)

print(employees[["name", "projects_completed", "performance_level"]])
```

	name	projects_completed	performance_level
0	Alice	5	Medium
1	Bob	9	High
2	Charlie	7	Medium
3	Diana	4	Medium
4	Evan	2	Low
5	Fiona	6	Medium
6	George	8	High

7	Hannah	3	Low
8	Ian	10	High
9	Jenna	5	Medium

14 Question 11 (10 Points)

Calculate Adjusted Salary

Write a function that calculates an adjusted salary for each employee based on their performance category. The function should take the original salary and the performance category as inputs and increase the salary by: 10% for 'High' performance. 5% for 'Medium' performance. No change for 'Low' performance. Use a loop to apply this function across the DataFrame and create a new column named `adjusted_salary`.

```
[73]: # Figure out the adjusted pay.
def get_adjusted_salary(salary, performance_level):
    if performance_level == "High":
        return salary * 1.10
    elif performance_level == "Medium":
        return salary * 1.05
    else:
        return salary

# Store adjusted salaries here.
adjusted_salaries = []

# Loop through the rows.
for i in range(len(employees)):
    current_salary = employees.loc[i, "salary"]
    current_performance = employees.loc[i, "performance_level"]

    new_salary = get_adjusted_salary(current_salary, current_performance)
    adjusted_salaries.append(new_salary)

# Add the new column.
employees["adjusted_salary"] = adjusted_salaries

print(employees[["name", "salary", "performance_level", "adjusted_salary"]])
```

	name	salary	performance_level	adjusted_salary
0	Alice	70000	Medium	73500.0
1	Bob	85000	High	93500.0
2	Charlie	80000	Medium	84000.0
3	Diana	75000	Medium	78750.0

4	Evan	68000	Low	68000.0
5	Fiona	72000	Medium	75600.0
6	George	78000	High	85800.0
7	Hannah	74000	Low	74000.0
8	Ian	77000	High	84700.0
9	Jenna	79000	Medium	82950.0